ASSIGNMENT 4 – CO4

CSA17 – ARTIFICIAL INTELLIGENCE

Learning: Algorithm Design, Implementation, Test, Debug and Evaluate

Prepared as a detailed answer set covering Tasks 1–6. The examples and code are illustrative, reproducible templates; actual execution results should be generated in the student's own Python environment before submission.

Contents

- Task 1 – Review and repair a learning protocol
- Task 2 – Decision-tree learning
- Task 3 – Gaussian-mixture EM and hidden variables
- Task 4 – Neural-network back-propagation
- Task 5 – Naive Bayes, kernel machines and explanation-based learning Task 6 – Reinforcement learning for an energy-aware controller

General assumptions

The assignment does not provide a single required dataset for every implementation task. Therefore, small transparent examples are used for manual traces and standard public datasets are suggested for model comparisons. Use fixed random seeds, keep training/validation/test data separate, and report measurements produced by the actual run. Never present a training score as a held-out test score.

TASK 1 – PSEUDOCODE REVIEW

1.1 Diagnose the submitted protocol

Deficiency	Why it is misleading
Training and testing use the same data	The model is evaluated on examples it has already seen, so memorisation can look like generalisation.
No held-out test set	There is no independent estimate of performance on unseen examples.
No validation or cross-validation	Hyper-parameters may be chosen without a fair model-selection process.
No named algorithm	"Suitable model" is ambiguous; another engineer cannot reproduce the experiment.
Accuracy is the only metric	With imbalanced classes, predicting the majority class can produce high accuracy while missing at-risk students.
No baseline	The result cannot be judged against a simple majority-class predictor.
No random seed or split policy	Repeated runs may differ, making comparisons unreliable.
No preprocessing policy	Scaling, imputation or feature selection performed before splitting can leak information.

1.2 Corrected protocol

ALGORITHM EvaluateStudentRisk(D)

INPUT: labelled dataset D; fixed seed s

OUTPUT: selected model, test metrics, baseline metrics

1. Remove duplicate records; define target and predictors.
2. StratifiedSplit D into TRAIN_VALID (80%) and TEST (20%), using seed s.
3. On TRAIN_VALID, create stratified 5 folds.
4. Candidate models := {LogisticRegression, DecisionTree(max_depth in {2,4,6}), RandomForest(n_estimators=200, max_depth in {4,8})}.
5. For each candidate and each fold:
   Fit preprocessing only on the fold's training portion.
   Fit candidate; predict the fold's validation portion.
   Record F1, recall, precision and ROC-AUC.
6. Select candidate with highest mean validation F1 (break ties using higher recall, then lower complexity).
7. Refit selected pipeline on all TRAIN_VALID.
8. Predict TEST exactly once.
9. Compute confusion matrix, accuracy, precision, recall, F1 and ROC-AUC.
10. Predict the majority class for every TEST example; compute the same metrics.
11. Return model, both metric sets and the fixed split information.

The test set is not used for feature selection, threshold selection or hyper-parameter tuning. If a decision threshold must be tuned, it is chosen using training/validation data only.

TASK 1 – DEFECT DEMONSTRATION

1.3 Example implementation and interpretation

A deliberately flexible decision tree can memorise a small training set. The following code compares resubstitution accuracy with stratified cross-validation and a final held-out test score. It uses a synthetic classification dataset so the demonstration is reproducible.

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split, cross_val_score from
sklearn.tree import DecisionTreeClassifier from sklearn.metrics import
accuracy_score import numpy as np


X, y = make_classification(
```

```
    n_samples=500, n_features=20, n_informative=3,
n_redundant=2, weights=[0.85, 0.15],     class_sep=0.7,
flip_y=0.08, random_state=17)


X_train, X_test, y_train, y_test = train_test_split(     X, y,
test_size=0.25, stratify=y, random_state=17)


model = DecisionTreeClassifier(random_state=17)  # unrestricted tree
model.fit(X_train, y_train)
train_acc = accuracy_score(y_train, model.predict(X_train)) cv =
cross_val_score(model, X_train, y_train, cv=5, scoring='accuracy') test_acc =
accuracy_score(y_test, model.predict(X_test))


print("Training accuracy:", round(train_acc, 3)) print("5-fold CV
mean:", round(cv.mean(), 3)) print("Held-out test accuracy:",
round(test_acc, 3)) print("Optimistic training gap:", round(train_acc-
test_acc, 3))
```

Expected interpretation

An unrestricted tree commonly achieves training accuracy close to 1.00 because it can form leaves around individual training records. Cross-validation and test accuracy are generally lower because those examples were not used to fit the corresponding tree. The exact numbers depend on the installed scikit-learn version and must be copied from the student's run, not invented.

A positive training-minus-test gap is the numeric overestimate. If the gap is zero or negative in a particular run, report that honestly and repeat with a more flexible tree or a noisier dataset while keeping the test protocol unchanged.

Conclusion

A valid learning protocol separates model fitting, model selection and final evaluation. The majority baseline and classsensitive metrics reveal whether the model is useful for identifying at-risk students.

TASK 2 – DECISION-TREE LEARNING

2.1 Model the requirement

Element	Definition
Input	Attendance (High/Low), assignment status (Done/Missed), internal marks (Good/Poor).
Output	Predicted result: Pass or Fail.
Objective	Learn a compact decision tree that predicts the result for a new student.
Hypothesis space	Finite trees whose internal nodes test one available categorical attribute and whose leaves predict a class.
Constraints	Only the three supplied attributes; small training set; avoid unnecessary depth and overfitting.
Decision at a node	Choose the attribute test that best separates the class labels in the examples reaching that node.
Required information	Class counts before and after each candidate split; entropy or Gini impurity.

2.2 Splitting hypothesis

At each node, select the unused attribute with the greatest information gain, defined as parent entropy minus the weighted entropy of its child partitions; this favours attributes that reduce class uncertainty.

2.3 Recursive construction pseudocode

FUNCTION BuildTree(S, A, defaultClass)

1. IF S is empty: RETURN Leaf(defaultClass).
2. IF all examples in S have class c: RETURN Leaf(c).

3. IF A is empty: RETURN Leaf(MajorityClass(S)).
4. FOR each attribute a in A:
   Compute weighted child entropy after splitting S by a.
   Gain[a] := Entropy(S) - weighted child entropy.
5. best := attribute with maximum Gain (fixed tie-breaking order).
6. Create node testing best.
7. FOR each value v of best:
   Sv := examples in S where best = v.
   child := BuildTree(Sv, A minus {best}, MajorityClass(S)).
   Attach child to node for branch v.
8. RETURN node.

The default class is the majority class of the parent node. It handles an unseen attribute value or an empty branch.

TASK 2 – MANUAL ENTROPY TRACE

2.4 Desk-check

No.	Attendance	Assignments	Internal	Result
1	High	Done	Good	Pass
2	High	Done	Poor	Pass
3	High	Missed	Good	Pass
4	High	Missed	Poor	Fail
5	Low	Done	Good	Pass
6	Low	Done	Poor	Fail
7	Low	Missed	Good	Fail
8	Low	Missed	Poor	Fail

There are 4 Pass and 4 Fail examples. Parent entropy = $-(4/8 \log_2(4/8) + 4/8 \log_2(4/8)) = 1.000$ bit.

Attribute	Child groups	Weighted entropy	Information gain
Attendance	High: 3P,1F; Low: 1P,3F	0.8113	0.1887
Assignments	Done: 3P,1F; Missed: 1P,3F	0.8113	0.1887
Internal marks	Good: 3P,1F; Poor: 1P,3F	0.8113	0.1887

For each attribute, $H(3,1) = -(3/4 \log_2 3/4 + 1/4 \log_2 1/4) = 0.8113$. Each attribute creates two groups of four, so weighted entropy is 0.8113 and gain is $1 - 0.8113 = 0.1887$.

All three attributes tie. With the stated tie-breaking order Attendance → Assignments → Internal, choose Attendance as the root. The High branch contains 3 Pass and 1 Fail; the Low branch contains 1 Pass and 3 Fail. Further splits are required.

Important branch decisions

For the High branch, splitting on Assignments or Internal produces one 2P/0F child and one 1P/1F child; weighted entropy is 0.5, giving branch gain $0.8113 - 0.5 = 0.3113$. The Low branch is symmetric and has the same gain. The next attribute can be selected using the same calculation.

TASK 2 – IMPLEMENTATION AND VERIFICATION

2.5 From-scratch split criterion

```
import numpy as np from collections
import Counter


def entropy(y):     counts = np.array(list(Counter(y).values()),
dtype=float)     probs = counts / counts.sum()     return -
np.sum(probs * np.log2(probs))


def gini(y):     counts = np.array(list(Counter(y).values()),
dtype=float)     probs = counts / counts.sum()     return 1.0 -
np.sum(probs ** 2)


def information_gain(y, groups):     total =
len(y)
   return entropy(y) - sum(len(g)/total * entropy(g) for g in groups)


# Demonstration on the assignment's eight records y =
["Pass","Pass","Pass","Fail","Pass","Fail","Fail","Fail"]
attendance = ["High","High","High","High","Low","Low","Low","Low"]
groups = [[y[i] for i in range(8) if attendance[i] == v]
        for v in ["High","Low"]]
print("Parent entropy:", round(entropy(y), 4))
print("Attendance gain:", round(information_gain(y, groups), 4)) print("Parent Gini:",
round(gini(y), 4))
```

Expected output for this desk-check: Parent entropy = 1.0000; Attendance gain = 0.1887; Parent Gini = 0.5000.

Verification against scikit-learn

```
from sklearn.datasets import load_breast_cancer from
sklearn.model_selection import train_test_split from
sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (accuracy_score, precision_score,  recall_score, f1_score,
confusion_matrix)


X, y = load_breast_cancer(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(
   X, y, test_size=.25, stratify=y, random_state=17) for
criterion in ["gini", "entropy"]:
   clf = DecisionTreeClassifier(criterion=criterion, random_state=17)
clf.fit(Xtr, ytr)     pred = clf.predict(Xte)
   print(criterion, accuracy_score(yte,pred),
precision_score(yte,pred),          recall_score(yte,pred),
f1_score(yte,pred))     print(confusion_matrix(yte,pred))
```

Use the same split for the from-scratch implementation and the library model. Report the actual printed metrics. A from-scratch learner should be checked for matching split rules, stopping conditions and tie-breaking; small differences can occur if these differ.

TASK 2 – OVERFITTING, PRUNING AND CHALLENGE

2.6 Full tree versus constrained tree

```
from sklearn.tree import DecisionTreeClassifier from
sklearn.metrics import accuracy_score for depth in
[None, 2, 4, 6]:
   tree = DecisionTreeClassifier(max_depth=depth, random_state=17)     tree.fit(Xtr, ytr)
   print(depth, tree.get_depth(), tree.get_n_leaves(),
accuracy_score(ytr, tree.predict(Xtr)),          accuracy_score(Xte,
tree.predict(Xte)))
```

For a proper pruning experiment, create a validation split from the training data. Choose max_depth or min_samples_leaf using validation performance, then refit on all training/validation data and evaluate the test set once. Record depth, leaves, training score, validation score and test score.

Model	Expected pattern	Interpretation
Unrestricted tree	Very high training score; potentially lower validation/test score	High variance and possible overfitting.
Depth-limited tree	Training score may decrease; validation may improve	Simpler model can generalise better.
Large minimum leaf size	Fewer small leaves; smoother boundaries	Reduces sensitivity to isolated records.

2.7 Change one value and predict

Change record 8 from Internal = Poor to Internal = Good while keeping its result Fail. Before recalculation, predict that the root tie will be broken: Attendance and Assignments still have the original 3P/1F versus 1P/3F partitions, while Internal marks becomes less balanced. Recompute gains to verify the prediction.

Identifier-like attribute challenge: add Student_ID with eight unique values. Information gain can become 1.0 because every unique ID isolates one record, producing pure singleton leaves. This is memorisation, not a meaningful predictive relationship.

Repair: remove identifiers from predictors; use minimum leaf size, depth limits, validation-based pruning and preferably gain ratio or a split criterion that penalises many-valued attributes. Always test on unseen students.

TASK 3 – HIDDEN VARIABLES AND EM

3.1 Locate the defect

The responsibility values r[i][k] are computed as unnormalised joint scores $\pi[k] \times \text{Normal}(x_i \mid \mu[k], \text{var}[k])$. They must be normalised across all K components for each observation i. Without normalisation, r[i][k] is not a posterior probability and the sum over k need not equal 1.

3.2 Small failing input

Point	x_i
1	0.0
2	1.0
3	4.0
4	5.0

Choose K = 2, $\pi = (0.5, 0.5)$, $\mu = (0, 4)$, and variances = (1, 1). For x = 0, the unnormalised scores are approximately $0.5 \times 0.3989 = 0.1995$ and $0.5 \times 0.000134 = 0.000067$. Their sum is 0.1996, not 1. The correct responsibilities are approximately (0.9997, 0.0003).

3.3 Minimal repair

```
FOR each observation i:
  FOR each component k:
    r[i][k] <- pi[k] * Normal(X[i] | mu[k], var[k])
  total <- SUM over k of r[i][k]
FOR each component k:      r[i][k]
<- r[i][k] / total
```

Only the normalisation step is added. The M-step then computes Nk, means, variances and mixture weights from valid posterior responsibilities.

TASK 3 – IMPLEMENTATION AND TESTS

3.4 Faulty and repaired EM

```
import numpy as np from
scipy.stats import norm


def em_1d(x, means, variances, weights, iterations=30, repair=True):
   x = np.asarray(x, float)
   means = np.array(means, float); variances = np.array(variances, float)     weights =
np.array(weights, float)     history, weight_sums = [], []     for _ in range(iterations):
r = np.column_stack([
         weights[k] * norm.pdf(x, means[k], np.sqrt(variances[k]))
for k in range(len(weights))])        if repair:
         r = r / np.maximum(r.sum(axis=1, keepdims=True), 1e-300)        Nk =
r.sum(axis=0)
      means = (r * x[:,None]).sum(axis=0) / np.maximum(Nk,1e-12)
      variances = (r * (x[:,None]-means)**2).sum(axis=0) / np.maximum(Nk,1e-12)        weights = Nk /
len(x)
      likelihood = np.sum(np.log(np.maximum(
         sum(weights[k]*norm.pdf(x,means[k],np.sqrt(variances[k]))                for
k in range(len(weights))), 1e-300)))
      history.append(likelihood); weight_sums.append(weights.sum())     return means,
variances, weights, history, weight_sums


x = [0., 1., 4., 5.]
bad = em_1d(x,[0.,4.],[1.,1.],[.5,.5],repair=False) good =
em_1d(x,[0.,4.],[1.,1.],[.5,.5],repair=True) print("Repaired mixing-
weight sum:", round(good[2].sum(),6)) print("Repaired final log-
likelihood:", round(good[3][-1],4))
```

The normalised version should keep the mixing weights summing to approximately 1.0. The faulty version may produce weights whose sum differs from 1 because its responsibilities are not posterior probabilities. In a robust implementation, also guard against zero component mass, variance collapse and underflow.

Required experiment

Run at least 20 initialisations with different fixed seeds. For each run store the log-likelihood at every iteration and the sum of weights. Plot both versions. The repaired EM algorithm should have non-decreasing observed-data log-likelihood up to numerical tolerance when each E-step and M-step is implemented correctly.

TASK 3 – REGRESSION CHECK AND DISCUSSION

3.5 Regression case

A case that may already work is a single-component mixture (K = 1). The only responsibility is 1 after normalisation, and the unnormalised responsibility is also harmless if $\pi[1] = 1$. The repaired version therefore preserves the expected update. x = np.array([1.0, 2.0, 3.0, 4.0])

```
r = np.ones((len(x), 1))  # K=1 posterior responsibilities
print("Each row sum:", r.sum(axis=1)) print("Mean:",
np.sum(r[:,0]*x)/np.sum(r[:,0]))
print("Variance:", np.sum(r[:,0]*(x- x.mean())**2)/np.sum(r[:,0]))
```

Expected output: each row sum is 1; mean is 2.5; population variance is 1.25.

Why the repair matters

- The E-step estimates the probability that component k generated observation i.
- Each observation must distribute exactly one unit of responsibility among all components.
- The M-step treats these values as fractional membership counts.
- Normalising per observation makes Nk meaningful and ensures mixture weights sum to one.

- A likelihood trace is a useful regression test: a decrease beyond numerical tolerance signals an implementation defect or numerical instability.

Implementation safeguards

Use log-sum-exp for component log probabilities on difficult data. Add a small variance floor to avoid zero variance. If Nk is nearly zero, reinitialise that component or stop with a clear diagnostic. Record convergence using the change in log-likelihood, not only a fixed iteration count.

TASK 4 – NEURAL NETWORK BACK-PROPAGATION

4.1 Define the state

For layer l, let n_l be the number of units, a^(0) = x R^(n₀), W^(l) R^(n_l × n_(l−1)), and b^(l) R^(n_l). The pre-activation ∈ ∈ ∈ vector is z^(l) = W^(l)a^(l−1) + b^(l), and activation is a^(l) = σ(z^(l)). The error term δ^(l) = ∂J/∂z^(l) has dimension n_l.

4.2 Forward and backward recurrences

Forward:
a(0) := x For l = 1,...,L:
z(l) := W(l) a(l-1) + b(l)
a(l) := sigma(z(l))

Backward for squared error J = 1/2 ||a(L)-y||²: delta(L) := (a(L)-y)
elementwise-multiplied by sigma'(z(L))
For l = L-1,...,1:
delta(l) := transpose(W(l+1)) delta(l+1)
elementwise-multiplied by sigma'(z(l)) dJ/dW(l) :=
delta(l) transpose(a(l-1)) dJ/db(l) := delta(l)

The output layer is the boundary condition because the loss is defined directly from its activation. Earlier errors depend on the next layer through the chain rule, so they are computed backward from L to 1.

4.3 Mini-batch training pseudocode

Initialise each W(l) with small random values; initialise b(l).
Repeat for epochs 1...E:
Shuffle training examples using fixed seed.
For each mini-batch B:
Set all gradient accumulators to zero.
For each (x,y) in B:
Run forward recurrence and calculate loss.
Run backward recurrence.
Add outer-product weight gradients and bias gradients.
Divide accumulated gradients by |B|.
Update W(l) := W(l) - eta * gradW(l).
Update b(l) := b(l) - eta * gradb(l).
Evaluate validation loss.
Stop if validation loss has not improved for patience epochs. Return weights from the best validation epoch.

TASK 4 – MANUAL 2-2-1 TRACE

4.4 Forward pass

Given x = (1, 0), y = 1, sigmoid activation, hidden weights (0.5, −0.4) and (0.3, 0.8), output weights (0.6, −0.2), zero biases, squared error J = 1/2(a−y)² and learning rate η = 0.5.

Hidden unit 1: $z_1 = 0.5(1) + (-0.4)(0) = 0.5$; $a_1 = \sigma(0.5) = 0.622459$.

Hidden unit 2: $z_2 = 0.3(1) + 0.8(0) = 0.3$; $a_2 = \sigma(0.3) = 0.574443$.

Output: $z_3 = 0.6(0.622459) - 0.2(0.574443) = 0.258225$; $a_3 = \sigma(0.258225) \approx 0.5642$.

Loss: $J = 0.5(0.5642 - 1)^2 \approx 0.0950$.

Backward pass

Output delta: $\delta_3 = (a_3 - 1)a_3(1-a_3) \approx (-0.4358)(0.2460) = -0.1072$.

Hidden deltas: $\delta_1 = (0.6\delta_3)a_1(1-a_1) \approx -0.0151$. $\delta_2 = (-0.2\delta_3)a_2(1-a_2) \approx 0.0052$.

Weight update

Output weights: $w_{31}(new) = 0.6 - 0.5(\delta_3 a_1) \approx 0.6333$; $w_{32}(new) = -0.2 - 0.5(\delta_3 a_2) \approx -0.1692$.

Input-to-hidden weights: $w_{11}(new) = 0.5 - 0.5(\delta_1 x_1) \approx 0.5076$; $w_{21}(new) = 0.3 - 0.5(\delta_2 x_1) \approx 0.2974$. Weights connected to x_2 do not change because $x_2 = 0$. Biases update by $b(new) = b - \eta\delta$ for their corresponding unit.

Values are rounded for readability; a code trace using full precision may differ in the last decimal place.

TASK 4 – NUMERICAL VERIFICATION AND XOR

4.5 NumPy implementation outline

```
import numpy as np sigmoid = lambda z:
1/(1+np.exp(-z)) def forward(x, W1, b1,
W2, b2):     h = sigmoid(W1 @ x + b1)
o = sigmoid(W2 @ h + b2)     return h, o


def loss(x, y, W1, b1, W2, b2):     _, o
= forward(x,W1,b1,W2,b2)     return
0.5 * np.sum((o-y)**2)


# Analytic gradients are obtained from the delta equations # on the previous
page. Compare each parameter gradient with:
# (loss(theta + eps) - loss(theta - eps)) / (2*eps) # Use
eps = 1e-5 and report max relative error.
```

Gradient checking compares analytic gradients with central finite differences. Define relative error as $|g_a - g| / \max(1, |g_a|, |g|)$ and report the maximum over all parameters. A small error (for example, below $1e{-}5$ in double precision) supports the implementation.

Perceptron versus multilayer network on XOR

x_1	x_2	XOR target
0	0	0
0	1	1
1	0	1
1	1	0

A single-layer perceptron cannot classify XOR because the positive and negative points are not linearly separable. A network with a nonlinear hidden layer can form a nonlinear decision boundary and learn the four examples.

```
# XOR data for an experiment
X = np.array([[0,0],[0,1],[1,0],[1,1]], dtype=float) y =
np.array([[0],[1],[1],[0]], dtype=float)
# Train a 2-hidden-unit sigmoid network with gradient descent. # Print
predictions after training and compare with targets.
```

Use multiple initialisations because a tiny XOR dataset can be sensitive to initial weights. Report the seed, learning rate, epochs and final predictions.

TASK 4 – STRESS EXPERIMENT

4.6 8×8 digits experiment

Use sklearn.datasets.load_digits. Scale pixel values to [0,1], use a stratified train/validation/test split, and compare hidden units {2, 8, 32, 64}, learning rates {0.001, 0.01, 0.1, 1.0} and activations {sigmoid, ReLU}. Select hyper-parameters using validation macro-F1 or validation loss; evaluate the selected configuration on the test set once.

```
from sklearn.datasets import load_digits from
sklearn.model_selection import train_test_split from
sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix
X, y = load_digits(return_X_y=True)
X = X / 16.0
Xtr, Xte, ytr, yte = train_test_split(
   X,y,test_size=.2,stratify=y,random_state=17)
# Build the requested NumPy network for each configuration.
# Record epoch loss, test accuracy, macro-F1 and confusion matrix.
```

Factor	Values	What to observe
Hidden units	2, 8, 32, 64	Capacity and overfitting.

Learning rate	0.001, 0.01, 0.1, 1.0	Slow learning, stable learning or divergence.
Activation	Sigmoid, ReLU	Saturation/vanishing gradients versus sparse activations.

To diagnose vanishing gradients, record the L2 norm of each layer's weight gradient per epoch. Very small early-layer norms while the output layer continues changing indicate weak gradient flow. A very large learning rate may cause loss spikes or divergence.

The assignment requests actual loss curves and metric values. Generate them from the implementation and insert the resulting plots and confusion matrix in the final submission; do not substitute guessed numbers.

TASK 5 – STATISTICAL LEARNING: NAIVE BAYES

5.1 Parameter estimation and classification

For class c and feature value v, estimate P(c) from class counts and P(x=v | c) from conditional counts. With Laplace $_j$ smoothing α, use P(x=v | c) = (N(c,v)+α)/(N(c)+α|V|). Predict the class with the largest log posterior.$_j$ $_j$

```
TRAIN_NAIVE_BAYES(D, alpha)
1. Count examples per class and estimate class prior.
2. For every feature j, class c and value v:
    conditional[c,j,v] :=
    (count(c,j,v) + alpha) /
    (count(c) + alpha * number_of_values(j)).
3. Store priors and conditional probabilities.

PREDICT(x)
1. For each class c:
    score[c] := log(prior[c]).
    For each feature j:
      score[c] += log(conditional[c,j,x[j]]).
2. Return class with greatest score.
```

Zero-count illustration

Example	Weather	Load	Class
1	Hot	High	Hazard
2	Hot	Low	Normal
3	Cold	Low	Normal

Without smoothing, P(Load=High | Normal) = 0/2 = 0. If a new record has Load=High, the Normal class posterior becomes zero even if other evidence favours it. With α=1 and two possible load values, P(High | Normal) = (0+1)/(2+2) = 0.25, so the model can still compare both classes.

Implementation experiment

Use a text dataset such as 20 Newsgroups or a tabular dataset with categorical features. Run α ∈ {0, 0.1, 1, 10}. For each setting, report held-out accuracy, precision, recall, F1 and log-loss. For α=0, handle unseen events carefully and document zero-probability behaviour.

TASK 5 – KERNEL MACHINES

5.2 Why a kernel model is needed

If hazardous and normal states are not linearly separable, a linear decision boundary may underfit. An SVM selects a separating boundary with a large margin. Support vectors are training observations closest to the boundary; they determine its position. A kernel function computes similarities as if data were mapped into a higher-dimensional feature space without explicitly constructing that mapping.

- Linear kernel: $K(x,z) = x^T z$; suitable when a linear boundary is adequate.
- RBF kernel: $K(x,z) = \exp(-\gamma\|x-z\|^2)$; can represent curved boundaries.
- C controls the penalty for margin violations; larger C fits training data more strictly.
- γ controls the locality of the RBF influence; too large can overfit and too small can underfit.

Experiment with 5-fold cross-validation

```
from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV, StratifiedKFold from
sklearn.pipeline import make_pipeline from sklearn.preprocessing import
StandardScaler from sklearn.metrics import accuracy_score, f1_score cv =
StratifiedKFold(n_splits=5, shuffle=True, random_state=17) linear =
GridSearchCV(
  make_pipeline(StandardScaler(), SVC(kernel="linear")),
```

```
    {"svc__C":[0.1,1,10,100]}, cv=cv, scoring="f1")
rbf = GridSearchCV(
    make_pipeline(StandardScaler(), SVC(kernel="rbf")),
    {"svc__C":[0.1,1,10,100],
     "svc__gamma":["scale",0.001,0.01,0.1,1]},     cv=cv,
scoring="f1") linear.fit(Xtr,ytr); rbf.fit(Xtr,ytr) for name,
model in [("linear",linear),("rbf",rbf)]:
pred=model.predict(Xte)     print(name,
model.best_params_,
        accuracy_score(yte,pred), f1_score(yte,pred))
    print("Support vectors:", model.best_estimator_[-1].n_support_)
```

Record training time using time.perf_counter around fit. Compare linear SVM, RBF SVM, Naive Bayes and the Task 2 decision tree on identical train/test indices. If the target is imbalanced, use macro-F1 or positive-class F1 consistently.

TASK 5 – EXPLANATION-BASED LEARNING

5.3 Domain theory and generalisation

A domain theory describes how observable operating conditions imply an overload hazard. The following example is a simple symbolic theory for the assignment; it is not an operational safety controller.

Example first-order rules

R1: HighCurrent(s) AND LowVoltage(s) -> ElectricalStress(s).
R2: ElectricalStress(s) AND HighTemperature(s) -> OverloadRisk(s).
R3: OverloadRisk(s) AND ProtectiveRelayFailed(s) -> Hazardous(s).
R4: OverloadRisk(s) AND ProtectiveRelayWorking(s) -> AlarmRequired(s).
R5: NormalCurrent(s) AND NormalVoltage(s) -> Stable(s).
R6: Stable(s) AND NormalTemperature(s) -> NormalState(s). R7:
Hazardous(s) -> DispatchInspection(s).

Explanation tree for one case

Suppose case s_1 has HighCurrent, LowVoltage, HighTemperature and ProtectiveRelayFailed. R1 derives ElectricalStress(s_1). R2 combines ElectricalStress and HighTemperature to derive OverloadRisk(s_1). R3 combines OverloadRisk and relay failure to derive Hazardous(s_1). R7 then derives DispatchInspection(s_1).

Generalised operational rule

The explanation supports the rule: HighCurrent(x) AND LowVoltage(x) AND HighTemperature(x) AND ProtectiveRelayFailed(x) → Hazardous(x). This rule retains the conditions necessary for the explanation while replacing the specific constant s_1 with a variable x.

Test on 20 designed cases

Create 20 records with combinations of the four predicates, including positive cases and near-miss cases where one required condition is absent. Apply both (a) the generalised rule directly and (b) the complete domain theory by forward chaining. Record correct classifications and number of rule applications/inference steps.

What was learned: the generalised rule is a compact operational summary of a proof path. It is not new factual knowledge if it is logically entailed by the domain theory; it is a compiled explanation that can reduce repeated inference.

TASK 6 – REINFORCEMENT LEARNING DESIGN

Stage 1: Interpret the problem

Element	Definition
Observation	Current occupancy, indoor temperature, outdoor temperature, electricity price, time slot and recent controller state.
Action	Off, Low or High cooling.

Objective	Minimise energy cost while maintaining comfort when occupied.
Constraints	Comfort temperature band during occupancy; actions limited to three settings; decisions every 15 minutes.
Episode	One operating day, beginning at the first time slot and ending after the final slot.
Decision	Choose an action using the observed state and learned value estimates.

Stage 2: Strategy choice

A fixed thermostat is a useful baseline but cannot adapt to changing prices. Imitation learning requires labelled actions from past operators. SARSA is on-policy and learns values for the policy that includes its exploration behaviour. A model-based method requires a reliable transition model. Q-learning is selected because the state and action spaces can be discretised, it learns from reward and next state, and it can learn an off-policy greedy policy while using exploration during training.

Stage 3: Critical information

Represent state as (occupancy bin, indoor-temperature bin, outdoor-temperature bin, price bin, time-of-day bin, previousaction bin). This is approximately Markov if these variables capture the important influences on the next room temperature and cost. If the environment has thermal memory not captured by the state, add recent temperature trend or previous indoor readings.

Actions are {Off, Low, High}. Reward is negative energy cost minus a comfort penalty when occupied: $r = -\text{price}\times\text{kWh} - \lambda\times\max(0, \text{distance}(\text{indoor_temp}, \text{comfort_band}))^2\times\text{occupancy}$. Choose λ large enough that comfort violations are meaningfully discouraged. Use ε-greedy exploration with ε decaying to a small floor; discount factor γ close to 1 because current cooling affects later comfort.

TASK 6 – Q-LEARNING PSEUDOCODE AND DESK-CHECK

Stage 4: Complete algorithm

```
ALGORITHM TrainEnergyController(episodes, alpha, gamma, epsilon) 1. Initialise
Q(s,a) := 0 for every discretised state and action.
2. FOR episode := 1 TO episodes:
    Reset simulator to start-of-day state s.
    WHILE episode is not terminal:
      With probability epsilon choose a random action.
      Otherwise choose action a with maximum Q(s,a).
      Apply a; observe reward r, next state s2 and terminal flag.        IF
terminal:          target := r        ELSE:
        target := r + gamma * MAX over actions b of Q(s2,b)
      Q(s,a) := Q(s,a) + alpha * (target - Q(s,a))
s := s2
    Reduce epsilon, but not below epsilon_min.
3. Extract policy pi(s) := argmax_a Q(s,a).
4. Evaluate pi on unseen days/seeds and compare with fixed thermostat.
5. RETURN Q and pi.
```

Stage 5: Four-update desk-check

Toy states: C = comfortable, W = warm, H = hot. Actions: Off and Cool. Use $\alpha = 0.5$, $\gamma = 0.9$, and initialise all Q values to zero. Assume the following observed transitions and rewards.

Update	State/action	Reward	Next state	Updated Q(s,a)
1	W / Cool	2	C	1.00
2	C / Off	1	C	0.50
3	H / Cool	3	W	1.50
4	W / Off	−2	H	−1.00

For update 1, target = $2 + 0.9\times\max Q(C,\cdot) = 2$; $Q(W,\text{Cool}) = 0 + 0.5(2-0) = 1$. For update 2, target = 1 and $Q(C,\text{Off}) = 0.5$. For update 3, target = 3 and $Q(H,\text{Cool}) = 1.5$. For update 4, target = −2 and $Q(W,\text{Off}) = -1$. The example uses zero next-state values at the time of each update.

TASK 6 – IMPLEMENTATION, ATTACK AND CONCLUSION

Stage 6: Simulator and evaluation

```
import numpy as np
ACTIONS = ["Off", "Low", "High"] def q_update(Q, s, a, reward, s2, alpha=.1,
gamma=.95, terminal=False):
   target = reward if terminal else reward + gamma*np.max(Q[s2])     Q[s,a] +=
alpha*(target-Q[s,a])


# Build a 15-minute simulator for a full day.
# State includes occupancy, indoor/outdoor temperature, price, # time slot and
previous action. Each action changes energy use # and indoor temperature;
occupancy determines comfort penalty.
# Train over episodes with epsilon-greedy exploration.
# Evaluate on separate random seeds and compare with a fixed rule.
```

For at least 20 seeds, report mean and standard deviation of smoothed episode reward, energy used (kWh), comfort-violation count and episodes to convergence. The fixed-rule baseline should use the same simulator days and starting conditions. Convergence can be defined in advance, for example, when the moving-average reward changes by less than a chosen tolerance over a fixed window.

Adversarial test

Construct a price pattern where electricity is cheap shortly before occupancy begins and expensive during occupancy. A reward that only penalises instantaneous energy cost may learn to keep the unit Off during the expensive occupied period, producing a low bill but poor comfort. Predict this failure before running. Then add an explicit occupancy-weighted comfort penalty and, if needed, a pre-cooling state feature so the controller can cool before occupancy.

Final conclusion

The assignment demonstrates the complete learning-engineering workflow: interpret the requirement, define the learning state or decision rule, design pseudocode, manually trace the logic, implement, instrument, challenge and validate. A model is not complete merely because it runs. It must be evaluated on held-out data, tested at boundaries, compared with a baseline and explained in terms of its assumptions and failure modes.

Submission checklist

- Insert actual execution outputs and plots for implementation tasks.
- Include source code files and record Python/library versions and random seeds.
- Use one consistent held-out split for fair model comparisons.
- Do not tune using the final test set.
- Explain any deviations from the illustrative manual calculations.